

A Naive Bayes Spam Filter for the TREC06 Corpus with Lambda Smoothing and Mutual Information Word Selection

Guy Ivan Ocon

National Graduate School of Engineering
Artificial Intelligence Program
University of the Philippines Diliman
Quezon City, Philippines
gdocon@up.edu.ph

Abstract—This paper describes a Naive Bayes spam filter written in Python without using any machine learning library. It was trained and tested on 37,782 email messages from the TREC 2006 public spam corpus, with 70% of the messages used for training and 30% for testing. Each message is treated as the set of words it contains, and the vocabulary built from the training set has 78,278 words. Without smoothing, the filter reached 99.94% precision but only 91.77% recall, because one word that was never seen in a class makes the probability of that class zero. Lambda smoothing was then tested with five values. The smallest value, 0.005, gave the highest precision at 99.98%. The largest value, 2.0, gave the highest recall at 98.79% and the highest F1 score at 98.91%. Following Hovold, mutual information was used to pick the 200 most informative words. With only these 200 words, the filter still reached 96.98% precision and 94.56% recall, and removing the 200 most frequent words before picking gave a small improvement. An extra experiment that used only the cleaned body of each message gave lower scores than using the whole message. Overall, the full vocabulary with a smoothing value of 2.0 gave the best F1 score.

Index Terms—Attribute selection, lambda smoothing, mutual information, naive Bayes, spam filtering, TREC06

I. INTRODUCTION

Spam detection in email can be treated as a classification problem with two classes. The first class is legitimate messages, called ham, and the second is unwanted messages, called spam [1]. A spam filter is trained on messages that are already labeled as ham or spam. It is then tested on messages that it has not seen before.

A filter can make two kinds of mistakes. It can let a spam message through, or it can mark a ham message as spam. The second mistake is usually worse, because the user may never see a real message. As Hovold notes, one is often willing to let more spam through in order to have fewer real messages marked as spam [2]. For this reason, this paper reports precision and recall separately. Precision shows how often a message marked as spam is really spam. Recall shows how much of the spam is caught.

This paper presents a Naive Bayes spam filter written from scratch in Python. It is tested on the Text REtrieval Conference (TREC) 2006 public spam corpus [3]. The work has three

parts. First, the classifier is built and tested without smoothing. Second, lambda smoothing is added, and five values of λ are compared. Third, the vocabulary is reduced to the 200 most informative words, which are chosen by mutual information (MI) as suggested by Hovold [2]. An extra experiment repeats the second and third parts using only the cleaned body of each message.

Section II describes the Naive Bayes algorithm. Section III describes the data and the implementation. Section IV presents and discusses the results, and Section V gives the conclusions and some ideas for future work.

II. THE NAIVE BAYES CLASSIFIER

A. The Naive Bayes Formula

The classifier chooses between two classes, ham and spam. Let ω stand for the class, and let $x_1, x_2, \dots, x_d$ be the words in the vocabulary V , where $d = |V|$ is the number of words. The Naive Bayes formula gives the probability of a class given the words [1]:

$$P(\omega \mid x_1, x_2, \dots, x_d) = \frac{\prod_{i=1}^d P(x_i \mid \omega) P(\omega)}{\sum_{\omega} \prod_{i=1}^d P(x_i \mid \omega) P(\omega)}. \quad (1)$$

Here $P(\omega)$ is the prior, which is the fraction of training messages that belong to class ω , and $P(x_i \mid \omega)$ is the likelihood of word x_i in class ω . The formula multiplies the likelihoods of the words. It treats the words as independent of each other once the class is known, which is why the method is called naive.

The bottom part of (1) is the same for ham and for spam. To classify a message, it is therefore enough to compute the top part for each class and pick the class with the larger value. In the implementation, only the words that appear in the message and are in the vocabulary are multiplied. Words that are not in the vocabulary are skipped.

B. Likelihood of a Word

The likelihood of a word in a class is the fraction of training documents of that class that contain the word [1]:

$$P(x_i | \omega) = \frac{\sum_{D \in D_\omega} I(x_i \in D)}{|D_\omega|}, \quad (2)$$

where $I(\cdot)$ is 1 if the word is in document D and 0 if it is not, D_ω is the set of training documents in class ω , and $|D_\omega|$ is the number of documents in that set. For example, if a word appears in 500 out of 1,000 spam documents, its likelihood in spam is 0.5. A word is counted only once per document, even if it is repeated many times in that document.

C. The Zero-Probability Problem and Lambda Smoothing

If a word never appears in a class in the training set, (2) gives it a likelihood of zero. Because (1) multiplies the likelihoods, one such word makes the whole value for that class zero, no matter what the other words are. If a message has one word that was never seen in spam and another word that was never seen in ham, both values are zero and there is nothing to compare. In that case, the classifier labels the message as ham.

A word that did not appear in a class during training can still appear in that class later [1]. Lambda smoothing handles this by adding a small number λ to every count:

$$P(x_i | \omega) = \frac{[\sum_{D \in D_\omega} I(x_i \in D)] + \lambda}{|D_\omega| + \lambda|V|}. \quad (3)$$

With $\lambda > 0$, no likelihood is exactly zero. One unseen word then lowers the value for a class but does not make it zero. When $\lambda = 1$, this is called Laplace smoothing [1]. When $\lambda = 0$, (3) is the same as (2).

D. Computation with Logarithms

A message can have hundreds of words, and each likelihood is a number smaller than one. Multiplying many small numbers gives a result that is too small for the computer to store, so it becomes zero. To avoid this, the classifier adds the logarithms of the prior and the likelihoods instead of multiplying them. The class with the larger sum is the same class that would have the larger product. A likelihood of zero becomes negative infinity in logarithm form.

When the actual probability in (1) is needed, the two sums are turned back into probabilities with the exponential function. The larger sum is first subtracted from both sums, which keeps the numbers from becoming zero again.

III. DATA AND IMPLEMENTATION

A. Data and Train-Test Split

The data come from the TREC 2006 public spam corpus [3]. The labels file lists 37,822 messages. Forty of these files were not found in the copy of the corpus used here, so 37,782 messages were used: 24,882 spam and 12,900 ham. Each file holds a whole email, including its header lines.

The messages were split into a training set with 70% of the messages and a test set with 30%. The split was done

TABLE I
NUMBER OF MESSAGES IN THE TRAINING AND TEST SETS

Set	Spam	Ham	Total
Training	17,417	9,030	26,447
Test	7,465	3,870	11,335
Total	24,882	12,900	37,782

separately for spam and for ham, so both sets have the same share of spam and ham. The messages were shuffled with a fixed random seed, so the same split comes out on every run. A check confirmed that no message is in both sets. Table I shows the sizes.

B. Tokenization and Vocabulary

Each file is read as a whole. A word is a sequence of letters that has a white space before it and a white space, a comma, or a period after it. The start and the end of the file count as white space. Under this rule, a word that touches a digit, a hyphen, or other punctuation is not counted. For example, in the text “Hello, world. Visit www.site.com or e-mail me@site.com for FREE money!” the words found are *Hello*, *world*, *Visit*, *www*, *or*, *for*, and *FREE*.

All words are changed to lowercase. The likelihood in (2) only asks whether a word is in a document or not, so each message is stored as the set of its different words. The vocabulary is the set of all different words in the training set, and it has 78,278 words. Of these, 15,829 appear only in spam messages and 46,814 appear only in ham messages. The words found in the most spam documents are *from*, *with*, *by*, *id*, and *for*. These are words from the header lines at the top of each email file.

C. Training and Classification

The classifier uses only the Python standard library. Training takes one pass over the training set. For each word, the number of spam documents and the number of ham documents that contain it are counted. The priors are $P(\text{spam}) = 0.6586$ and $P(\text{ham}) = 0.3414$. The likelihoods are computed with (2) when there is no smoothing and with (3) when there is smoothing, and they are stored as logarithms.

To classify a message, its words are found in the same way as in training. The logarithm of the prior and the logarithms of the likelihoods of its words are added for each class, as described in Section II-D. The message is labeled spam only if the spam sum is larger than the ham sum. Otherwise it is labeled ham. This includes the case where both sums are negative infinity. As a quick check, the made-up message “Congratulations, you won free money. Claim your prize now.” was classified as spam by the classifier without smoothing, with a spam probability of 0.996. With $\lambda = 2.0$, the probability is 0.99998.

Smoothing was tested with $\lambda = 2.0, 1.0, 0.5, 0.1$, and 0.005 . The value of λ with the highest F1 score was used in the next steps.

D. Choosing Informative Words

Some words, such as *a*, *of*, and *the*, are common in both classes and say little about whether a message is spam [1]. Hovold [2] ranks words by the MI between the presence of a word and the class:

$$MI(X; C) = \sum_{x \in \{0,1\}} \sum_{\omega \in C} P(x, \omega) \log \frac{P(x, \omega)}{P(x)P(\omega)}, \quad (4)$$

where C is the set of classes, $X = 1$ if the word is in the message, and $X = 0$ if it is not. A word gets a high score if knowing whether it is in a message tells a lot about the class. A word that is about equally common in spam and in ham gets a score near zero.

The probabilities in (4) are computed from the priors and from the smoothed likelihoods in (3) with the best value of λ . Words that appear in fewer than three training documents are not ranked, which follows Hovold's advice to drop very rare words [2]. The 200 words with the highest scores are kept. Each one is called a spam word if its likelihood is higher in spam, and a ham word if its likelihood is higher in ham. The classifier is then trained again using only these 200 words, so that $|V| = 200$ in (3), and it is tested on the same test set.

Hovold got better results from MI when the 200 most frequent words were removed before ranking [2]. A second selection was therefore made. The 200 words that appear in the most training documents are removed first, and the top 200 words by MI are taken from the words that are left.

E. Cleaned Message Bodies

An extra experiment, which is not required by the assignment, uses a cleaner version of each message. Only the body of the email is read, not the header lines. If the email has a plain-text part, that part is used. If it has only a Hypertext Markup Language (HTML) part, the HTML tags, scripts, and styles are removed first. A short list of words is then removed. The list has very common words, such as *the*, *and*, and *for*, and leftover web and HTML words, such as *www*, *http*, *com*, and *href*. Words shorter than 3 letters or longer than 20 letters are also removed, and so are words that appear in fewer than three training documents. The smoothing test and the 200-word selections are then repeated with the same model. The classifier without smoothing was not run on cleaned bodies.

One more way of picking 200 words was tried in this experiment. For each word, a spam rate and a ham rate are computed. The spam rate is the number of spam documents that contain the word plus 1, divided by the number of spam documents plus 2. The ham rate is computed in the same way from the ham documents. Adding 1 and 2 keeps a rate from being zero. The words are then ranked by how far apart the two rates are. The 100 top words with a higher spam rate and the 100 top words with a higher ham rate are kept. This is called the document-frequency selection in the tables.

F. Evaluation Measures

Spam is the positive class. A true positive (TP) is a spam message classified as spam. A true negative (TN) is a ham

TABLE II
TEST RESULTS BY SMOOTHING PARAMETER (%)

λ	FP	FN	Precision	Recall	F1	Accuracy
None	4	614	99.94	91.77	95.68	94.55
0.005	1	813	99.98	89.11	94.23	92.82
0.1	6	492	99.91	93.41	96.55	95.61
0.5	33	173	99.55	97.68	98.61	98.18
1.0	54	114	99.27	98.47	98.87	98.52
2.0	73	90	99.02	98.79	98.91	98.56

message classified as ham. A false positive (FP) is a ham message classified as spam, and a false negative (FN) is a spam message classified as ham. Precision and recall are computed as [1]

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}. \quad (5)$$

Precision is the share of messages marked as spam that are really spam. Recall is the share of all spam messages that were caught. The F1 score combines the two into one number:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 TP}{2 TP + FP + FN}. \quad (6)$$

For example, at $\lambda = 2.0$ the test set gives $TP = 7,375$, $FP = 73$, and $FN = 90$, so $F_1 = 14,750/14,913 = 98.91\%$. Accuracy is the share of all test messages that were classified correctly. When precision and recall point to different settings, the F1 score is used to choose between them.

IV. RESULTS AND DISCUSSION

A. Classification Without Smoothing

Without smoothing, the classifier reached a precision of 99.94% and a recall of 91.77% (first row of Table II). Only 4 of the 3,870 ham messages in the test set were marked as spam, but 614 of the 7,465 spam messages were missed.

Both numbers come from the zero-probability problem in Section II-C. A message can be labeled spam only if none of its words has a zero likelihood in spam. Few ham messages pass this strict condition, so there are very few false positives. The same problem lowers recall. In 632 of the 11,335 test messages, both classes had a probability of zero. These messages could not be compared, so they were labeled ham, and any spam message among them was missed.

B. Effect of the Smoothing Parameter

Table II and Fig. 1 show the results for the five values of λ . As λ goes from 0.005 to 2.0, recall goes up from 89.11% to 98.79%, and precision goes down from 99.98% to 99.02%. In terms of messages, the number of missed spam messages drops from 813 to 90, while the number of ham messages marked as spam grows from 1 to 73.

No single value is the best for both measures. The best precision comes from $\lambda = 0.005$, and the best recall comes from $\lambda = 2.0$. The smallest value behaves much like the classifier without smoothing, with very high precision but low recall.

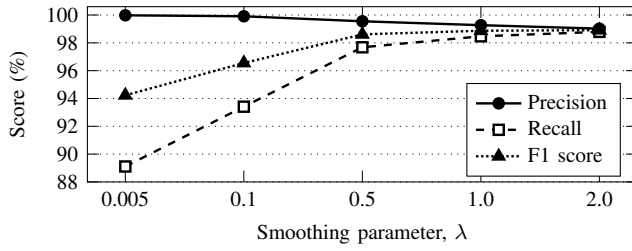


Fig. 1. Precision, recall, and F1 score on the test set for the five values of λ . The horizontal axis is not drawn to scale.

TABLE III
TEST RESULTS ON FULL MESSAGES AT $\lambda = 2.0$ (%)

Vocabulary	Size	FP	FN	Precision	Recall	F1
All words	78,278	73	90	99.02	98.79	98.91
MI	200	220	406	96.98	94.56	95.75
MI ^a	200	209	391	97.13	94.76	95.93

^aRanked after the 200 most frequent words were removed.

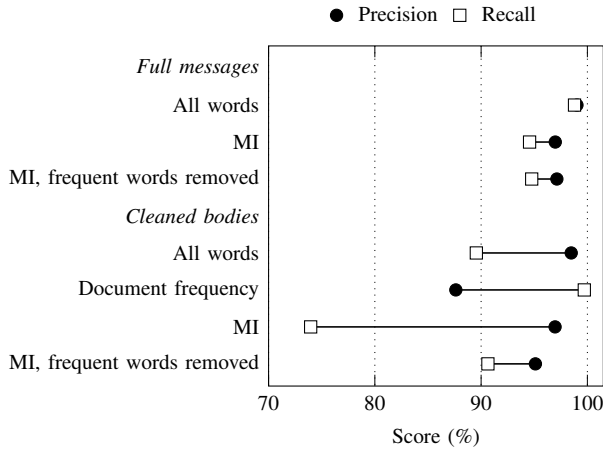


Fig. 2. Precision and recall for the seven setups in Tables III and IV at $\lambda = 2.0$. Each row is one setup. The line between the two marks shows the gap between precision and recall.

Because precision and recall disagree, the F1 score was used to decide. The best F1 score, 98.91%, comes from $\lambda = 2.0$, which also has the best accuracy at 98.56%. This value was used in the next experiments. The result for $\lambda = 1.0$ is very close, with an F1 score of 98.87%. Compared with no smoothing, $\lambda = 2.0$ catches 524 more spam messages, at the cost of 69 more ham messages marked as spam.

C. Classification with a 200-Word Vocabulary

Table III and Fig. 2 show the results when only the 200 words with the highest MI are used. The F1 score went down from 98.91% to 95.75%, and the number of wrongly classified test messages went up from 163 to 626. Still, 200 words are a very small part of the full vocabulary of 78,278 words, and the classifier keeps most of its performance with them.

Of the 200 words, 127 are spam words and 73 are ham words. Some of the spam words come from the email header

TABLE IV
TEST RESULTS ON CLEANED BODIES AT $\lambda = 2.0$ (%)

Vocabulary	Size	FP	FN	Precision	Recall	F1
All words ^a	28,084	103	780	98.48	89.55	93.80
Document frequency	200	1,052	22	87.62	99.71	93.27
MI	200	173	1,944	96.96	73.96	83.91
MI ^b	200	348	699	95.11	90.64	92.82

^aWords that occur in at least three training documents.

^bRanked after the 200 most frequent words were removed.

lines and not from the message text, such as *mime*, *esmtip*, *outlook*, and *mimeole*. Other spam words are about the usual topics of spam, such as *stock*, *offer*, *diet*, and *investors*. The ham words include *thanks*, *anyone*, and *problem*, and also names such as *handyboard*, *dmdx*, and *lugnet*.

When the 200 most frequent words were removed before ranking, 128 of the 200 selected words stayed the same and 72 were new. The new words include *money*, *shares*, *sensor*, and *linux*. The results improved a little. The F1 score went from 95.75% to 95.93%, with 11 fewer false positives and 15 fewer false negatives. This agrees with Hovold's finding that removing the most frequent words first helps [2], although the gain here is small.

D. Results on Cleaned Message Bodies

After cleaning, the vocabulary has 28,084 words. The five values of λ were tested again, and $\lambda = 2.0$ again gave the best F1 score. Table IV shows the results. With the full cleaned vocabulary, recall is 89.55% and the F1 score is 93.80%. Both are lower than the results on full messages in Table III, so cleaning did not help.

Two facts from the experiment help explain this. First, 2,555 of the 11,335 test messages (22.5%) had no words left after cleaning. For these messages, the classifier has only the prior to use. Second, Section IV-C showed that some of the most informative words come from the header lines. Cleaning removes the header lines, so these words can no longer be used.

The 200-word vocabularies also did worse on cleaned bodies. With MI, only 58 of the 200 words are spam words and 142 are ham words, and recall fell to 73.96%. Removing the 200 most frequent words first raised recall to 90.64%, which is a much bigger improvement than on full messages. The document-frequency selection caught almost all spam, with a recall of 99.71%, but it marked 1,052 ham messages as spam, so its precision is only 87.62%.

E. Limitations

This work has three limitations. First, the best value of λ was chosen using the results on the test set, so the reported scores may be a little higher than they would be on new messages. Second, all results come from one 70/30 split. A different split could change the small differences, such as the one between $\lambda = 1.0$ and $\lambda = 2.0$. Third, only five values of λ were tested, and the best one was the largest. A value larger than 2.0 might do even better.

V. CONCLUSION AND FUTURE WORK

This paper presented a Naive Bayes spam filter written from scratch and tested on the TREC 2006 public spam corpus. The best setup used the full vocabulary of 78,278 words with $\lambda = 2.0$. It reached a precision of 99.02%, a recall of 98.79%, and an F1 score of 98.91%.

Three conclusions can be drawn from the experiments. First, smoothing is needed. Without it, the classifier was very precise but missed 614 spam messages, because one unseen word was enough to make a class probability zero. With smoothing, larger values of λ gave higher recall and slightly lower precision. Second, a vocabulary of only 200 words chosen by MI still gave an F1 score of about 96%, and removing the 200 most frequent words before choosing gave a small improvement. Third, reading only the cleaned body of each message gave worse results than reading the whole message.

The classifier can be improved in several ways. The value of λ should be chosen on a separate validation set and not on the test set. Values of λ larger than 2.0 should be tried, since the best value found here was the largest one tested. Other vocabulary sizes between 200 words and the full vocabulary could be tested to see how many words are really needed. Finally, because cleaning removed useful header words, a future version could clean the body of the message but keep the header lines.

REFERENCES

- [1] P. Naval and A. Labao, "AI 201 programming assignment 1: Naive Bayes spam filter," Course handout, Univ. of the Philippines Diliman, Quezon City, Philippines, 2026.
- [2] J. Hovold, "Naive Bayes spam filtering using word-position-based attributes," in *Proc. 2nd Conf. Email Anti-Spam (CEAS)*, Stanford, CA, USA, 2005.
- [3] G. V. Cormack, "TREC 2006 spam track overview," in *Proc. 15th Text REtrieval Conf. (TREC)*, Gaithersburg, MD, USA, Nov. 2006, NIST Special Publication 500-272.